Making Multimodal Melodies: Design and Implementation of a Comprehensive, Interactive Music Visualization System

Kalu James Christopher Obasi

Abstract

This thesis presents the design and implementation of an interactive music visualization system that bridges auditory and visual modalities through user-driven input in real time. Unlike traditional visualizers that passively interpret pre-recorded audio, this system enables dynamic, tactile interaction using a MIDI controller, an Arduino microcontroller, and an LED display panel. Through a combination of hardware and software components, the system translates musical attributes such as pitch, dynamics, tempo, and pedaling into vivid and customizable light displays. The design emphasizes accessibility and modularity, supporting various MIDI devices and allowing users to engage at both beginner and advanced levels of musical experience. The system offers a novel platform for exploring the relationship between sound and sight, with potential applications in performance, education, and accessible music interaction.

Acknowledgments

I could easily fill several dozen pages of this thesis report with expressions of gratitude to all the people who have helped me with its creation. There are some people, however, who have been so instrumental that this work would be rendered impossible without their assistance, and I feel they must be mentioned here for this thesis to be truly complete.

First among these are my mother, father, and older brother, who have given me so much support throughout my education at Princeton University. I must also thank my independent work advisors—Dr. Radhika Nagpal, Dr. Elizabeth Margulis, Irene Sha, and Allison Chen—for their advice and mentorship at all stages of the development of this report.

In addition, I must express my gratitude to all the professors and researchers at Princeton who have inspired me on this journey. These include, but are by no means limited to, Dr. Ruha Benjamin, Dr. Matthew Clayton, Dr. Marcel Dall'Agnol, Dr. Akil Fletcher, Dr. Jesse Gomez, Dr. Casey Lew-Williams, Dr. Andrés Monroy-Hernández, Dr. Ian Peebles, Dr. Adrienne Raphel, and Dr. Christina Riehl. A special mention must be given to Dr. Alexandros Chatzigeorgiou and Dr. Apostolos Ampatzoglou at the University of Macedonia, who graciously offered me a rewarding research internship in their Department of Applied Informatics in the summer of 2024.

Furthermore, I would like to extend my appreciation to the many undergraduate student organizations at Princeton in which I found a lifelong community of friends, scholars, and artists. These include, but are again not limited to, the Princeton Pianists Ensemble, the Princeton University Glee Club, the Princeton African Students Association, the students and staff of Mathey College, the Princeton Katzenjammers, the Princeton Caribbean Connection, the National Society of Black Engineers, and the Black Arts Company. Finally, I want to thank those brave, kind souls who provided their feedback on the drafts of this report: Madison Anderson, Christine Chen, Bianca Del Campo, and Chern Yang.

Thank you all very much. I sincerely hope that you find the following literature compelling enough to be worthy of all your guidance and encouragement.

Contents

1 Introduction **5**

2 Background and Related Work **7**

2.1 Music Visualization 7

2.2 The MIDI Protocol 9

2.3 Interactive Art 10

2.4 Musical Glossary 11

3 Approach **13**

4 Design and Implementation **14**

4.1 The MIDI Controller 14

4.2 The Arduino Microcontroller 16

4.3 The Arduino IDE 17

4.4 The Hairless MIDI to Serial Bridge 17

4.5 The loopMIDI Virtual Port and Auditory Output 18

4.6 The LED Panel 19

5 Full Functionality of the System **20**

5.1 Basic Tactile Interaction: Keys 20

5.2 Brightness: Pressure Pads and Potentiometers 21

5.3 Light Duration: Sustain Pedal 22

5.4 Musical Coloration: Keys and Pressure Pads 23

6 Evaluation: Successes, Limitations, and Possible Revisions **25**

6.1 Tempo and Articulation 25

6.2 Dynamics 25

6.3 Musical Key 26
6.4 MIDI Compatibility 28
6.5 User Feedback 29

7 Applications and Future Work **31**

8 Conclusion **32**

A Appendices **37**

A.1 Code 37
A.2 Photos and Videos 37

B Honor Code **37**

1 Introduction

Advancements in digital technologies have broken down many existing barriers between sensory modalities. Music, once considered an exclusively auditory experience, is now frequently augmented by visual, tactile, and haptic stimuli in a range of applications. This expansion of multimodal interaction has created new possibilities for how music can be composed, performed, understood, and experienced. In particular, real-time music visualization systems have emerged as a compelling avenue for both artistic expression and creative exploration, offering a synesthetic experience to performers and audiences alike.

This thesis presents a novel design and implementation of one such system: a multimodal, interactive platform that translates live musical input into dynamic visual patterns on a custom-built LED panel. Unlike most existing music visualizers, which passively analyze pre-recorded audio tracks, this system is designed to respond to real-time tactile interaction from the user. The result is an audiovisual experience in which the physical actions of the performer are translated into vibrant, animated displays of light. In addition, these displays are not merely decorative, but structured to convey musically significant information to system users, including, but not limited to, rhythm, tempo, articulation, and volume.

Throughout this report, my aim is to situate this project within the broader context of music visualization, human-computer interaction, and educational technology, while also reflecting on the creative and technical decisions that shaped its development. My goal is not only to document a novel interactive system with which to engage in music, but also to explore the broader implications of combining visual and auditory modalities—particularly how such systems might democratize musical performance and composition, support multisensory learning techniques, and provide more inclusive pathways to experience sound in general.

The work presented here builds upon an independent project that I completed in the spring of 2024 entitled "Harmonizing Sound and Light: MIDI-Enabled LED Visualization System." That project introduced a basic prototype for a real-time music visualizer and served as a foundational study of the feasibility for such systems, which provided valuable insights into user interaction and technical constraints. However, the prior implementation was limited in scope, with reduced functionality and minimal support for expressive control. In contrast, the present thesis represents a significant extension of that earlier work, introducing an expanded platform for musical interactivity with many more features. Where relevant, I will describe which aspects of the earlier system have been retained and which have been revised or entirely redesigned. However, in general, to minimize confusion and highlight the strengths of the system in its most recent iteration, I assume that the reader is largely unfamiliar with the earlier project.

2 Background and Related Work

My system's design leverages the Musical Instrument Digital Interface (MIDI) protocol to make some improvements on current applications of music visualization. In the following section, I provide what I believe to be the most relevant contextual information on these topics so that my explanations of the system's functionality later in this report are more clearly understood.

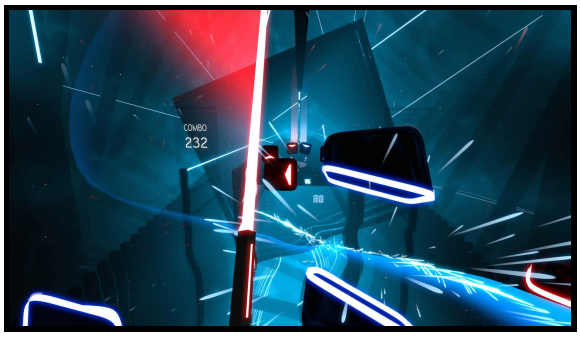
2.1 Music Visualization

Music visualization refers to the process of generating animated imagery based on a piece of music. These visualizations are typically rendered in real time alongside the accompanying music, so that users receive visual feedback in tandem with auditory feedback. The most effective music visualization techniques attain a high level of correlation between the music's characteristics and the components of the visual image being displayed, such that musical tracks that differ in aspects like tempo, instrumentation, and volume create animations that differ in aspects like coloration, orientation, and speed.

Music visualization is a very broad category of media, and the techniques therein are quite varied and used in a wide range of applications. Many popular examples, such as Ryan Geiss' MilkDrop and the Apple iTunes Visualizer, are aimed at general audiences to enhance their music listening experience [1], [2]. Other techniques are tailored toward more specific user bases. For instance, video games like Beat Games' *Beat Saber* and Nintendo's *Rhythm Heaven Groove* weave music and sound effects into the core gameplay mechanics, appealing to music listeners and video game players alike [3], [4].



(a) An image from Apple's music visualizer, available in iTunes for Mac devices. According to the lower left corner, though it is hard to see, this image was produced by Bruno Mars' 2012 single "Locked Out of Heaven."



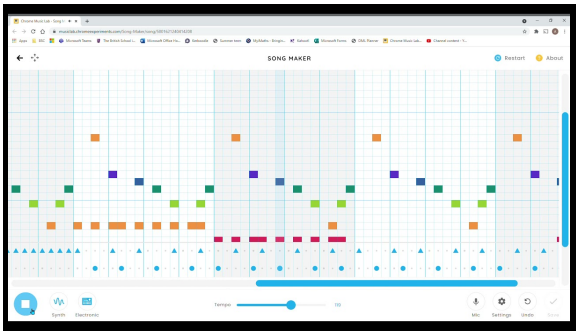
(b) A gameplay screenshot from *Beat Saber*, released for virtual reality devices in 2018. The aim of this game is to use two sabers to cut cubes that float toward the player in time with the rhythm of the background music.

Figure 1: Two well-known and widely used music visualization applications.

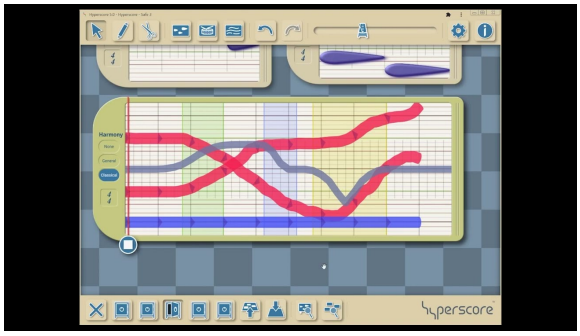
One through line among these applications is that they all tend to largely separate users from the performance of the music. These visualizations usually correspond to some sort of background music that viewers and listeners have little or no part in creating. Even in the rhythm games mentioned above, the player is not actually making any of the music. Rather, they are using the preset soundtrack of the game to influence their gameplay decisions. Overall, this makes for a potentially interesting but very passive music-listening experience. This is a key point of motivation for my work, as I intend to put users in more direct control of the visualization process with multiple points of interaction and forms of feedback.

To their credit, many music visualization applications do give users a more active role, particularly those made for educational purposes. For example, Google's Chrome Music Lab is a free website with a series of musical "experiments" to teach people about music and sound. These include the Song Maker, in which users click boxes on the screen to compose simple melodies, and the Shared Piano, in which they type on their computer keyboard to play a virtual piano with one another via the Internet [5]. Hyperscore, a software tool first developed in the Media Lab at the Massachusetts Institute of Technology, takes a similar approach. The program is designed to introduce students to writing music by allowing them to draw lines in a graphical environment that

are then mapped to musical structures to create a composition [6].



(a) The Song Maker application in the Chrome Music Lab.



(b) The graphical interface presented to users in Hyperscore.

Figure 2: Two applications of music visualization for education.

In implementations like these, however, the interactive components are heavily simplified in order to teach people about music in a user-friendly way. Users are not really "playing a musical instrument" in the traditional sense; they are typing, clicking, drawing shapes, or doing another basic task that results in (usually quite simplistic) music. In the development of my system, my aim is to help users interact with music visualization in both simple and complex ways, depending on their own familiarity with aspects of music theory, composition, and performance.

2.2 The MIDI Protocol

MIDI is a digital language created in the 1980s that allows electronic musical instruments and other devices to communicate and exchange musical information. The purpose of this language is to fully standardize the way musical data is encoded and transmitted, making it possible to connect all the different devices needed for a performance or composition in a simple and intuitive way [7]. Despite a fair amount of criticism directed at MIDI devices for their inability to express all the same things as acoustic instruments [8], the protocol quickly exploded in popularity, to the extent that MIDI became the widely accepted standard for artists, record producers, audio engineers, and other creators in the music industry. Today, MIDI is integrated into the vast majority of modern

electronic music devices; one would be very hard pressed to find a synthesizer, sequencer, or digital audio workstation (DAWs) without any MIDI functionality made in the last 10 or 20 years.

Given its utility and popularity across music technology, it may not come as a surprise that the MIDI language has found notable use in music visualization applications. MIDI has been used in conjunction with visualization software to examine the melodic progression of musical pieces, compare structural differences between different musical genres, and propose new forms of musical notation in two-dimensional and three-dimensional space, among many other purposes [9], [10], [11]. Again, though, these MIDI visualizations tend to be of pre-recorded musical excerpts, not of real-time musical input from a performer or group of performers, which motivates my design of a more interactive approach to the medium.

2.3 Interactive Art

Some applications of music visualization, including the one I designed for this thesis, can be classified as interactive works of art. The term "interactive artwork" refers to a piece of art in which there exists an area for viewers to perform some series of actions and in which those specific actions have specific effects on certain aspects of the art. In other words, an interactive artwork is only fully realized when the viewer offers their own participation [12] [13]. What counts as "interaction" on the part of the viewer varies greatly from piece to piece. In some examples like Random International's *Rain Room*, it simply involves moving around the space and watching how the art adapts to one's position or orientation [14]. In others like *Six-Forty by Four-Eighty* by Jamie Zigelbaum, it involves touching some part of the artwork itself [15]. I believe that my thesis falls into the latter category.

Some interactive art pieces appeal to some area of music or sound. One clear example is the Jam-O-Drum, a tabletop device that uses computer imagery and velocity-sensitive input terminals

to create a collaborative music-making experience. With the capacity for up to six participants to play music together, this system serves as an exercise in social and musical improvisation. Although my system only supports one primary user at a time, it shares some characteristics with the Jam-O-Drum regarding interactive auditory art [16].

2.4 Musical Glossary

Naturally, I use a fair amount of musical key[1] terms throughout this report. As such, to assist readers who may not have a background in music theory or performance, I have provided a glossary with brief definitions of the most frequently referenced musical terms, as well as some terms that are not explicitly written in this report but would still be helpful to know.

- **Note:** A single musical tone, such as the sound of a single piano key being pressed.
- **Chord:** A group of notes played at the same time to create musical harmony.
- **Pitch:** The perceived "highness" or "lowness" of a note. For example, a soprano singer produces higher-pitched notes than a bass singer.
- **Musical key:** A group of notes centered around a "home" note called the tonic. The key defines which notes are typically used in a piece and influences its overall sound or mood.
- **Major key:** A type of musical key that typically sounds bright, happy, or triumphant. Major keys follow a specific pattern of whole and half steps between notes and are the most common in Western popular and classical music.
- **Minor key:** A type of musical key that typically sounds darker, sadder, or more somber than a major key. Minor keys use a different pattern of intervals than major keys, often giving them a more emotionally complex or introspective tone.
- **Tonic:** The central note of a musical key, around which all other notes in that key are organized. It often feels like the "resting place" of a piece of music.

[1] Pun *absolutely* intended.

- **Key signature:** A notational symbol at the beginning of a piece that indicates which notes will be consistently sharp or flat. It helps define the musical key.
- **Tempo:** The speed at which music is played, usually measured in beats per minute (BPM). Faster tempos create more energetic music; slower tempos feel more relaxed.
- **Dynamics:** Variations in volume during a musical performance, ranging from very soft to very loud.
- **Velocity (MIDI):** In the context of MIDI, velocity refers to how forcefully a key is pressed. Higher velocities generally correspond to louder sounds for electronic MIDI instruments.
- **Sustain pedal:** A foot pedal (typically on a piano or MIDI keyboard) that causes notes to continue sounding even after the keys are released. Pedaling is often used during musical performances to connect notes or create resonance.
- **Articulation:** The way individual notes are played—for example, smoothly connected (legato) or sharply separated (staccato). Pedaling and timing can influence articulation.
- **Pentatonic scale:** A musical scale with five notes per octave, commonly used in many world music traditions and popular songs.
- **Chromatic scale:** A twelve-note scale made up entirely of half-steps (semitones), including all possible pitches within an octave.
- **Octave:** A musical interval between one note and another with double its frequency. For example, the note C played on one part of a keyboard and the next higher C sound an octave apart. Notes an octave apart sound similar but higher or lower in pitch.

3 Approach

There were a few core principles that informed my initial design of the system, even before implementing any features with actual hardware or software. Above all else, I felt that the system must be intuitive to use and that the most basic functions of its interface must be immediately obvious, even to users with little knowledge of music. Future adaptations of the system might be more tailored to musicians with a certain level of experience in either composition or performance, but this initial one should be as simple as possible.

In addition to being simple, the process of interacting with the system should still be reasonably interesting. Visual output should employ a variety of animations and colors to keep users engaged and curious, and it should clearly and vividly capture multiple different aspects of music performance at once, making it suitable for those without a music background and those *with* one.

The system should also be fairly light on hardware components. One could imagine a version of this system that uses much more complicated and sophisticated musical information processing, with signals running back and forth between multiple MIDI devices or even multiple computers. However, for the purposes of simplicity and approachability, I attempted to maximize the capabilities of the system while using only one fairly small and fairly basic MIDI controller. Again, future adaptations of the system might aim for a much larger and more involved experience to increase user interaction.

4 Design and Implementation

I implemented my design concept for a music visualization system using a series of interconnected hardware and software components. In the following section, I will explain these components and their functions within the system, along with relevant contextual information where applicable.

4.1 The MIDI Controller

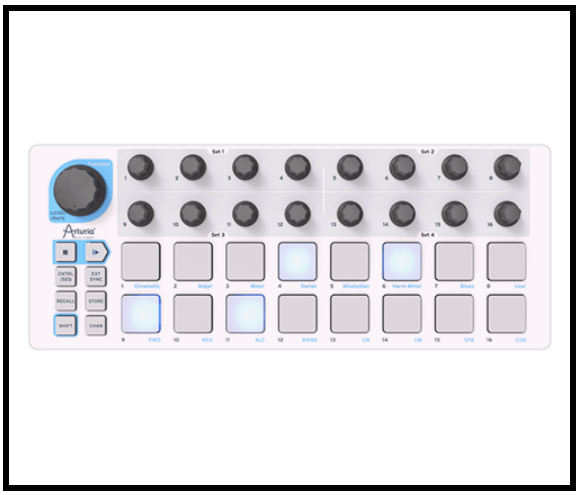
Simply put, a MIDI controller is an interactive device that transmits MIDI to nearby devices. A MIDI controller is the core of my music visualization design, serving as the key point of interaction between the user and the rest of the components. Importantly, a MIDI controller does not usually produce sound of any kind on its own and should not be thought of as a self-contained electronic musical instrument. Rather, the data sent by MIDI controllers are used by other devices or applications to create music.

MIDI controllers are designed with a range of different interface controls or actuators, including standard input buttons, pressure pads, rotary encoders, potentiometers, levers, faders, and more. This makes MIDI controllers capable of sending a wide variety of data to other devices via, in turn, a wide variety of MIDI signals. The specific pattern of input from the user and the resulting pattern of MIDI signals sent from the controller dictate the overall behavior of the system.

The MIDI protocol is generally very simple and flexible across different types of MIDI controllers. In fact, the system I designed is theoretically compatible with any MIDI controller (albeit with varying results and to varying degrees of quality, which will be discussed later in this report). Here, I chose a MIDI keyboard controller, I deem the best choice for this working system, as it communicates its capabilities more clearly than many of the other available options. The Arturia Keystep 37 is the MIDI controller that I used most often during the system's development process, and it appears frequently in my documentation photos and videos.



(a) The Arturia KeyStep 37, a fairly standard MIDI keyboard controller [17]. Each key on the keyboard has a specific MIDI pitch value, which is sent in a MIDI signal when that key is pressed.



(b) The Arturia BeatStep, a MIDI controller that uses square pressure pads for input [18]. Like the KeyStep 37, each pad on the BeatStep sends a specific MIDI signal when pressed down.

Figure 3: Two different MIDI controllers made by the music electronics company Arturia. The keys on the KeyStep 37 and the pads on the BeatStep transmit the same type of MIDI signal, meaning they may share similar levels of compatibility with the overall system. In my view, however, the system's functionality is easier to communicate with a keyboard controller.

Many MIDI controllers are compatible with a selection of external peripherals that are connected directly to them to produce certain effects. The most important example for the purposes of this system is the sustain pedal. When the sustain pedal is pressed down on an acoustic piano, it moves the dampers away from the piano strings, letting them freely vibrate and produce sound until the vibrations naturally subside or the pedal is released. Electronic MIDI sustain pedals mimic the behavior of their acoustic counterparts by sending a unique type of MIDI signal called a MIDI Control Change message, which tells other devices like audio synthesizers or virtual instruments whether to sustain notes.

```
+1.118 - MIDI In: Ch 1: Controller 64 value 127
+1.248 - MIDI In: Ch 1: Controller 64 value 0
+1.554 - MIDI In: Ch 1: Controller 64 value 127
+1.759 - MIDI In: Ch 1: Controller 64 value 0
```

Figure 4: An example of the MIDI Control Change messages sent by an electronic MIDI sustain pedal is activated. Most MIDI pedals transmit signals with only two velocities: 127 when it is pressed down and 0 when it is released. Some more advanced models can send intermediate values when the pedal is only partially pressed.

4.2 The Arduino Microcontroller

Similar to my previous music visualization prototype, this design uses a single Arduino microcontroller to relay data from the MIDI controller to other components of the system [19]. Since this program is more complex and more susceptible to bugs and glitches, I thought it might be helpful to use a more complex microcontroller. As such, I tested a few different Arduinos to see which one resulted in the smoothest real-time performance.

To confirm whether a different microcontroller was truly necessary, I started my new design with the same board used in the previous design: the Arduino Uno R3 [20]. To my surprise, the Uno R3 remained suitable for the system in its early stages of development. However, as the development continued, the software used for the system became increasingly complicated and took up more and more of the board's memory, resulting in many problems. There would sometimes be significant latency between the input from the user and the output from both the visual and auditory feedback mechanisms, and the system would routinely crash if too many elements on the MIDI controller were activated at once. To ensure the system's ability to operate under pressure, I decided to find a board with more advanced specifications.

The next board I tried to implement was the Arduino Mega 2560, which came to mind first because of its better flash memory and static rapid-access memory (SRAM) [21]. This change did result in some notable improvements. The increased memory made the system much less susceptible to crashes, even under the pressure of many different MIDI events being executed at a rapid rate. Since the clock speed of the Mega 2560 is identical to that of the Uno R3, however, the same speed limitations and latency issues persisted throughout. The Mega 2560 is also a much larger microcontroller than the Uno R3, and since the system only requires the use of two input/output pins on the board, I found that its increased size was not worth its moderate performance boost. Similar sentiments could be shared about the Arduino Due, which I also tried very briefly [22].

The last board I tried, and the one that I found to be the most useful, was the Arduino Uno R4. With its higher clock speed and even greater SRAM, this board provided clear performance advantages over the Mega 2560 while maintaining much of the same ease of use and low power consumption of the Uno R3 [23], [24]. Any prior issues exhibited by the system in terms of transmission latency or signal loss disappeared entirely when incorporating the Uno R4. In addition, the Uno R4 WiFi[2] comes with a built-in LED matrix, which can be easily programmed for various functions with a native library in the Arduino IDE. Although the Arduino matrix is not really part of the visual feedback of the system, I still found it to be a helpful component in the design process, using it to display important messages to aid in feature implementation and debugging.

4.3 The Arduino IDE

Naturally, the use of an Arduino microcontroller requires the use of a script written for the controller using the Arduino IDE. This script, entitled "PlayThePlatform," handles the flow of information between the MIDI controller and the LED panel, as well as some other software applications that will be described in the following subsections. The most relevant Arduino libraries for this system are the MIDI Library to regulate MIDI input and output and the FastLED Library to control the state of the LEDs. As of this writing, both libraries are compatible with any Arduino architecture. This means that the overall functionality of this system is theoretically reproducible on all Arduino boards and is not strictly limited to the Uno R4.

4.4 The Hairless MIDI to Serial Bridge

While the Arduino IDE has a native library for MIDI signal processing, most Arduino boards, including the Uno R4, do not support direct input from an external MIDI controller via a USB

[2]The Arduino Uno R4 comes in two variants, namely the ‘Minima’ model and the ‘WiFi’ model. The two variants share nearly identical specifications, but the latter model is slightly more expensive and comes with added WiFi and Bluetooth capabilities and a built-in LED matrix. Although I did not find these additional features to be explicitly necessary, I still used the WiFi model in my design; however, the system should remain fully functional with either.

port. To avoid this issue, this system utilizes a software application called the Hairless MIDI to Serial Bridge. As the name suggests, this application can route MIDI signals from a MIDI device to a serial device using USB ports on a computer [25]. For the purposes of this system, this application runs on a desktop or laptop computer and routes signals from the MIDI controller to the Arduino board and, in turn, the LED panel for visual output. The software also has the option to display incoming and outgoing MIDI messages, which was a vital asset to me as I tested the system functionality with different MIDI controllers.

4.5 The loopMIDI Virtual Port and Auditory Output

As noted above, most MIDI controllers do not produce any sound on their own. Therefore, this system requires the use of an external application to read MIDI signals and produce sound instead. The easiest way to do this, in my view, is by using the loopMIDI application, which creates a virtual MIDI port through which MIDI signals can be transmitted [26]. In this system, the Hairless MIDI to Serial Bridge not only routes signals from the MIDI controller to the Arduino board, but also routes those same signals from the board to a virtual loopMIDI port.

After being sent through the loopMIDI port, these MIDI signals can be read by another application to create music. Almost any MIDI-compatible software application works for this task, so the decision to choose one specific application for this system was entirely subjective. I chose the Chrome Music Lab's MIDI software for many of the system's documentation videos as it was easy to set up and produced a nice, simple piano sound. A user with a certain level of experience with MIDI could easily reconfigure the system using a different audio application and create different musical effects to suit their own needs.

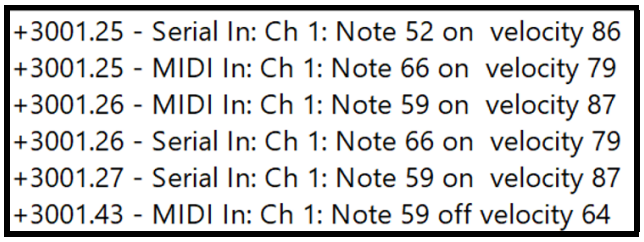


Figure 5: The Hairless MIDI to Serial Bridge, set up to route signals from an Arturia KeyStep 37 MIDI controller to an Arduino board and then to a virtual loopMIDI port. "Note On" signals are sent when a key on the MIDI controller is pressed, and "Note Off" signals are sent when a key is released. The velocity of the signal, recorded between 0 and 127, corresponds to the strength or gentleness with which the key was pressed. "Note Off" signals always have a velocity of 64.

4.6 The LED Panel

A group of 176 WS2812B RGB LED lights serves as the visualization component of this system. This panel gets input from the Arduino and is powered by an external battery. This is a similar approach to the visual component for my previous independent work, but there is one very important design change. In the previous system, the LEDs were arranged in a single continuous strip [19]. The idea was that this strip could be woven into certain shapes or patterns, adding a three-dimensional look to the visual display. During the development process, however, I found that this effect took more away from the system than it added. For example, user interaction with the system would often illuminate LEDs on a part of the strip that the user could not see, thereby not providing the user with much visual feedback at all.

To solve this problem in the newer iteration, I chose to cut the LED strip into smaller sections and join them with a series of solderless connectors to make a single unified panel of LEDs. This makes all the LEDs visible to the user, keeping them fully aware of the effects that their actions have on the system. The LEDs are also arranged in a spiral shape of sorts in order to communicate visual information about musical pitch, with higher notes corresponding to LEDs closer to the center.

5 Full Functionality of the System

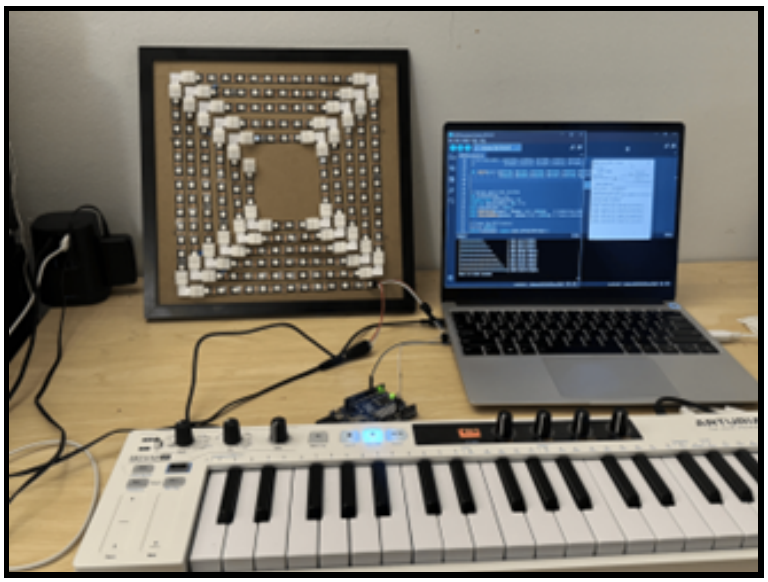


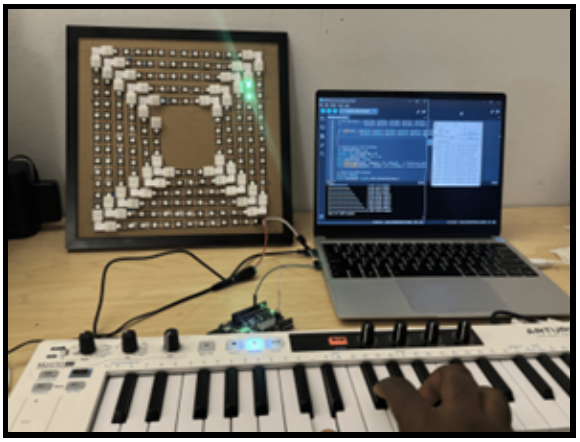
Figure 6: All parts of the music visualization system and their connections. As displayed here, the primary point of interaction with the system is a MIDI controller—in this case, an Arturia KeyStep 37. This controller is connected to a laptop computer supporting the loopMIDI virtual port, the Hairless MIDI to Serial Bridge, and the digital audio application. Also connected to the laptop is an Arduino Uno R4, which relays signals between the computer and the panel of LEDs. The external power source of the LED panel is not visible in this picture, nor is the electronic sustain pedal connected to the MIDI controller.

This section of the report will thoroughly explain all the features of the music visualization system, both in terms of how users can interact with the system and what events result from doing so. Importantly, not every MIDI controller may be able to use the system for its full functionality; cases in which a specific interface control is necessary to utilize a certain feature will be noted as such.

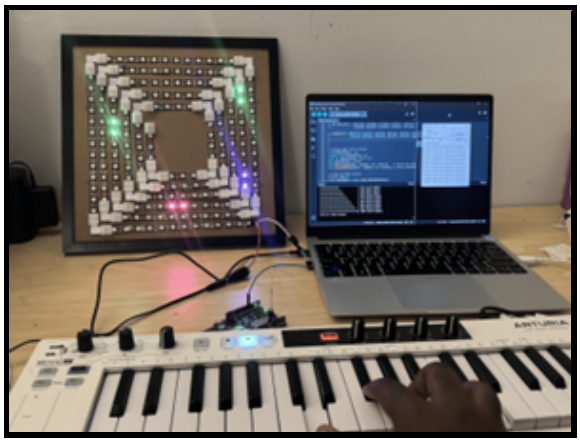
5.1 Basic Tactile Interaction: Keys

When a user presses a key on the key bed of the MIDI controller, two adjacent bulbs light up on the panel. Each key on the keyboard is mapped to a certain area of the LED strip such that higher musical pitches illuminate bulbs closer to the center of the panel. When the user releases this

key, the illuminated bulbs are turned off. As was the case in the previous version of this system, this version supports multiple keys being pressed simultaneously and in rapid succession to create colorful animations on the LED panel that relate directly to the tactile input from the user. Because this feature does not require any interface controls apart from the keys, it can be used with even the simplest MIDI keyboard controllers.[3]



(a) The pattern of light on the panel that results from a user pressing down the D3 key on the controller.



(b) The pattern of light that results from a user pressing down the D3, F#3, A4, and D4 keys simultaneously.

Figure 7: Light patterns produced by individual (a) and chordal (b) MIDI input.

5.2 Brightness: Pressure Pads and Potentiometers

As mentioned in an earlier section, the WS2812B LED architecture allows the brightness of the LEDs to be changed. This system responds to user input via a velocity sensitivity algorithm such that the strength with which the key is pressed corresponds to the brightness of the LEDs. In other words, more forceful key presses by the user result in brighter lights on the panel, and softer presses result in dimmer lights. This means that the volume or dynamics of the music can be represented in a visual format on the LED panel. Naturally, this effect is difficult to demonstrate using pictures alone, so a link to an online repository with videos of this feature (and all others in the system) has been provided in the appendix of this report.

[3] Kindly excuse the camera quality in the photographs of the system. For convenience, they were taken with my phone camera, which is not excellent.

If the MIDI controller used for the system has pressure pads, the velocity-sensitive brightness feature of the system can be toggled on and off. Although velocity-sensitive brightness is activated by default, pressing a pad on the controller will turn it off, making the panel brightness static and unresponsive to musical dynamics. In this state, though, the brightness can still be raised and lowered by twisting a potentiometer on the MIDI controller. Pressing the same pad again will put the velocity sensitivity back in effect.

Proper utilization of this feature requires the keys on the MIDI controller to be sensitive to velocity. Given their utility in musical expressiveness, velocity-sensitive keys have been considered a standard feature on most MIDI keyboard controllers for many years. However, some entry-level or toy-grade controllers may still not have them, rendering this feature inapplicable in some cases.

5.3 Light Duration: Sustain Pedal

If the MIDI controller used in the system is compatible with a sustain pedal, as most modern controllers tend to be, the system can produce a unique effect on the LED panel. When a user holds down the pedal and presses a key, the LED panel illuminates as usual. However, if that user releases the key while still holding down the pedal, the lights corresponding to the key will remain illuminated until the pedal is released. This creates a visual representation of pedaling in musical performance that reflects the typical auditory representation.

Acoustic pianos are typically designed with two other pedals in addition to the sustain pedal. The soft pedal gives the piano sound a quieter, altered tone, and the sostenuto pedal allows certain keys to be sustained selectively (as opposed to sustaining all of them at once like the normal sustain pedal does). Electronic MIDI peripherals that mimic the effect of these other pedals do exist, but they are usually expensive and difficult to find. In addition, these other pedals are really only compatible with keyboard controllers, so features activated by them would only be usable with

keyboards; and furthermore, many keyboard controllers only support the sustain pedal, rendering the other two essentially useless. Therefore, I chose not to design any effects in this system for the soft pedal or the sostenuto pedal.

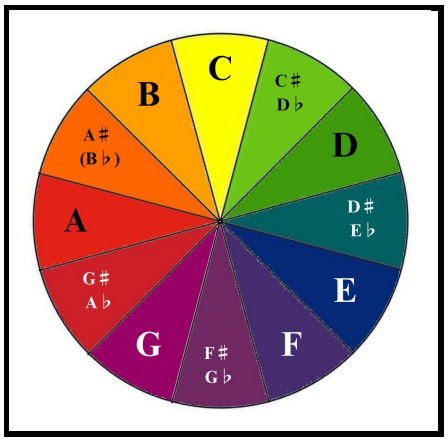
5.4 Musical Coloration: Keys and Pressure Pads

The color of the LEDs illuminated when a key is pressed corresponds directly to the associated musical note of that key. This gives each key its own color and, in turn, each musical chord its own color palette. The system supports two different mapping algorithms between musical notes and colors based on well-known scales frequently used in music composition. These can be toggled using a pressure pad on the MIDI controller, provided that the controller has such pads.

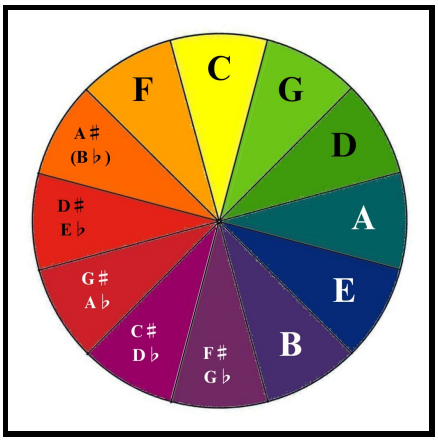
The first algorithm maps a 12-tone color wheel to the chromatic (12-note) musical scale.[4] With this mapping, adjacent keys on the keyboard have similar colors of light associated with them.[5] For example, the D key creates green light, and the neighboring C# and D# keys create similar shades of yellow and green light, respectively. This approach is used by many other music visualization applications, including my own previous independent work. In my view, this is a more intuitive mapping than the others, as it makes the relationship between music and color easy to understand, even for users without a strong musical background. The second algorithm maps the same 12-tone color wheel to the pentatonic (five-note) musical scale. This may be a less easily understood way to connect music and color than the previous method, as it means that adjacent keys will have quite different colors associated with them. However, it may appeal to users with a moderate to high level of musical aptitude.

[4]The specific logic for connecting colors to musical notes is somewhat arbitrary. For example, C did not *have* to be yellow for any reason. It just made some level of sense to me.

[5]The hexadecimal color codes on the wheels presented in this figure are the same ones programmed into the Arduino software for the system. However, WS2812B are not always *excellent* at representing the exact shades or hues programmed into them, especially when it comes to very dark or highly saturated colors. As such, there are some mild discrepancies between the colors in this report and the colors in the photographs of the real system.

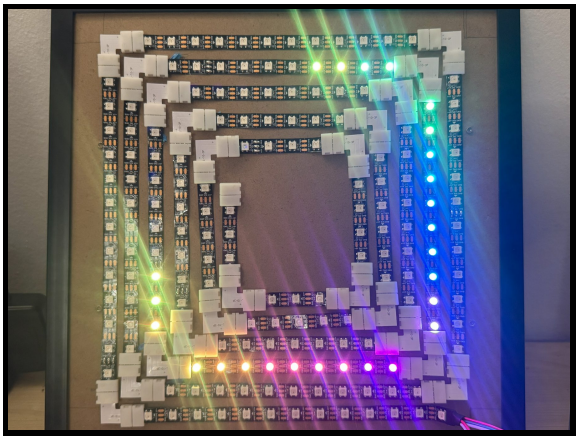


(a) A diagram showing which note values are mapped to which colors using the pentatonic scale algorithm.

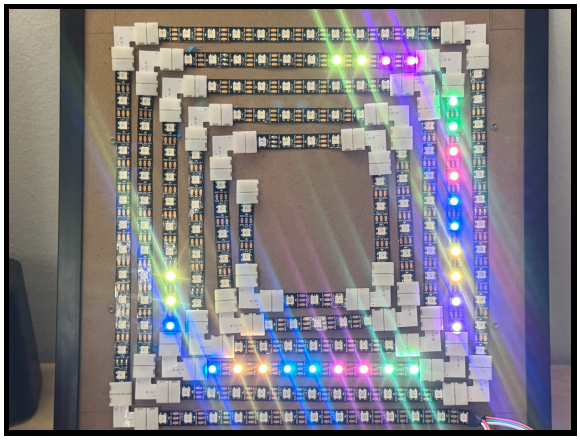


(b) A diagram showing which note values are mapped to which colors using the pentatonic scale algorithm.

Figure 8: Graphical representations of the two musical coloration algorithms. Incidentally, 6 of the 12 musical notes are mapped to the same color in both the chromatic scale mapping and the pentatonic scale mapping.



(a) The panel after 13 adjacent keys are pressed and sustained with the pedal, using the chromatic scale coloration algorithm.



(b) The panel after 13 adjacent keys are pressed and sustained, using the pentatonic scale coloration algorithm.

Figure 9: The two different coloration algorithms. Each pair of LEDs with the same color represents one keyboard key. The same 12 colors are represented in both images, but the associated colors of adjacent keys are more similar with the chromatic coloration than the pentatonic coloration.

6 Evaluation: Successes, Limitations, and Possible Revisions

I chose to evaluate the design and implementation of my system primarily based on its ability to present various musical properties in an interactive visual format. In large part, the system is capable of expressing musicality in a wide range of forms and caters to the musical aptitude levels of many different types of users. However, there are still some notable aspects of music performance that are not as well represented as others, which could be given more attention in future iterations. I also provide some points of evaluation that have less to do with the musicality and more with other areas of its design.

6.1 Tempo and Articulation

In my view, the tempo of music is the most well-visualized musical characteristic. The lights on the LED panel turn on and off in exact time with the user pressing the MIDI controller keys. This creates a clear visualization of how fast or slow the musical piece is, as more rapidly changing animations on the LED panel correspond directly to faster tempos.

The sustain pedal provides additional information for visualization. Two renditions of the same piece might sound very different depending on whether or not certain notes are sustained by the pedal, even keeping most other musical variables like tempo the same in both. The LED panel captures these differences in articulation by suspending some lights when the pedal is pressed down. With these features implemented, someone watching the LED panel while another person interacts with the MIDI controller can get a solid impression of the music's speed and dramatic interpretation.

6.2 Dynamics

Dynamic contrast is another musical characteristic that the system clearly visualizes. The dynamics of a piece of music refers to the variations in volume as the piece is played. Piano performances

often entail significant dynamic contrasts in which pianists press the keys with varying levels of force to produce louder or softer musical phrases.[6] The MIDI velocity sensitive feature of my system successfully visualizes these dynamic changes, both for people actively using the system and people passively watching it in operation.

One slight limitation of this feature is that while the colors of the lights on the LED panel are individually addressable, their brightness levels are not. There is a single 'brightness' value that applies throughout a whole LED strip at once, which means that different bulbs cannot be illuminated with different brightness levels at the same time. This seems to be an intrinsic property of the WS2812B LED architecture, which I was unable to easily modify.

I can see this limitation being avoided in a future extension of this system by using a different type of LED strip called SK6812. These LEDs use an RGBW color system, which is very similar to RGB but adds an additional value for pure white light. This not only makes SK6812 better for creating certain pale or muted light colors than WS2812B but also makes the brightness of each LED individually addressable simply by raising or lowering its W value.

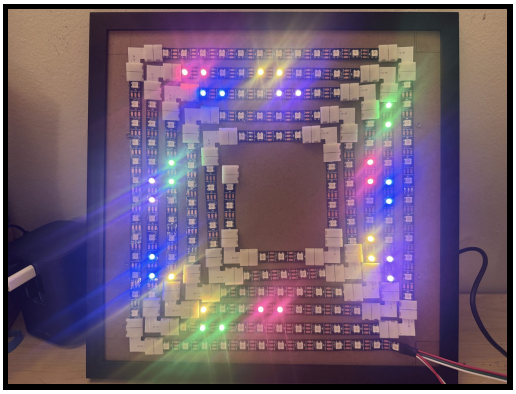
6.3 Musical Key

To my surprise, the system is able to incorporate the key signatures of certain musical pieces into its visualization. This comes from the algorithm that maps the colors of light to the pentatonic scale. Many popular excerpts of music rely heavily on the major pentatonic scale to create melodies and harmonies.[7] When these pieces are played on the keyboard controller while the pentatonic scale coloration is in effect, the color palette of the LED display changes clearly depending on the key signature in which the piece is played. This is technically observable with the colors mapped to the chromatic scale as well, but it is much less obvious, as demonstrated in the following figures.

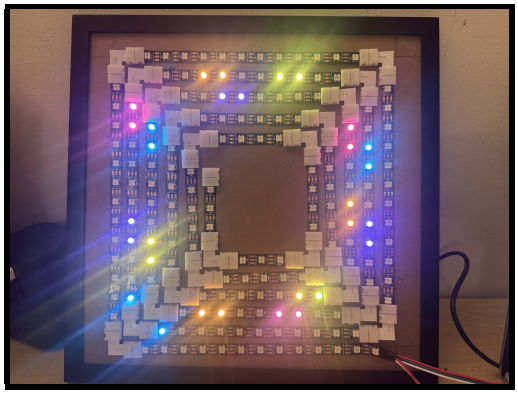
[6] In fact, the instrument's full name, "pianoforte," refers to its ability to play both soft (piano) and loud (forte) sounds. This was the main improvement that the piano made over its predecessor, the harpsichord.

[7] While there are countless examples of this, two that come to mind quickly are "Amazing Grace" and "Auld Lang Syne."

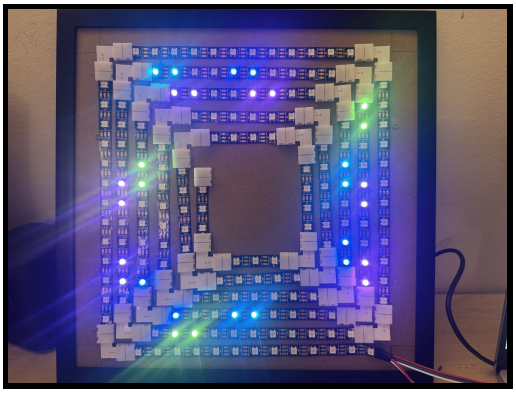
Although this is not a very robust or consistent way to visualize key signatures in music, it does present a basis upon which future versions of this system could be built. One could imagine a more intelligent system that learns from common patterns of tactile interaction to create a distinct color palette for each key signature or that is able to more closely distinguish between major key signatures and minor key signatures.



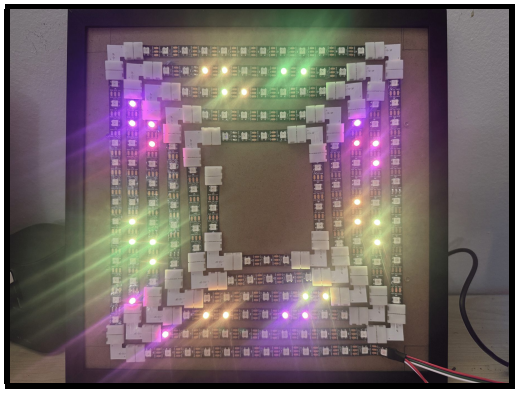
(a) 15 keys of the D major pentatonic scale sustained on the panel using the chromatic scale coloration.



(b) 15 keys of the A-flat major pentatonic scale sustained on the panel using the chromatic scale coloration.

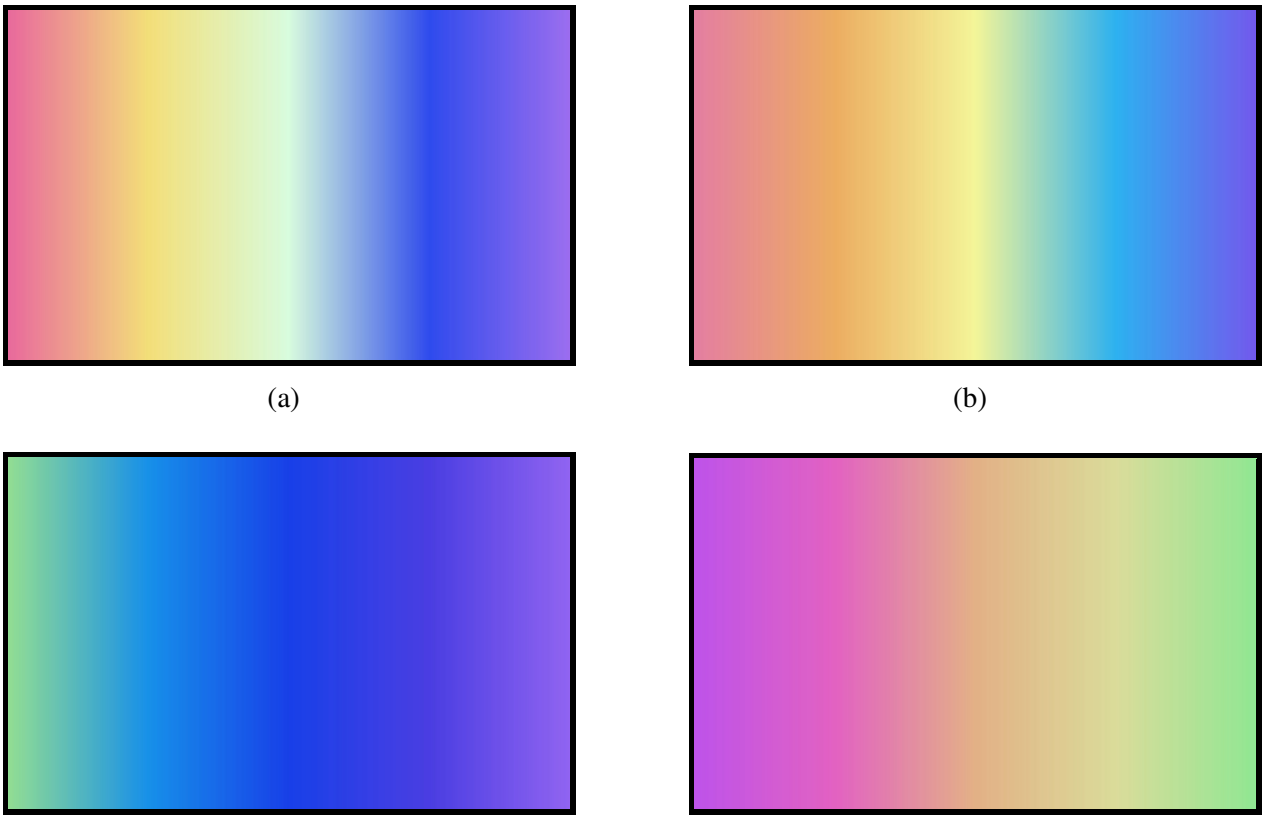


(c) 15 keys of the D major pentatonic scale sustained on the panel using the pentatonic scale coloration.



(d) 15 keys of the A-flat major pentatonic scale sustained on the panel using the pentatonic scale coloration.

Figure 10: These photographs show that the effect of the key signature on the colors of the visualization is more vivid and noticeable with the pentatonic scale coloration (bottom) than with chromatic scale coloration (top).



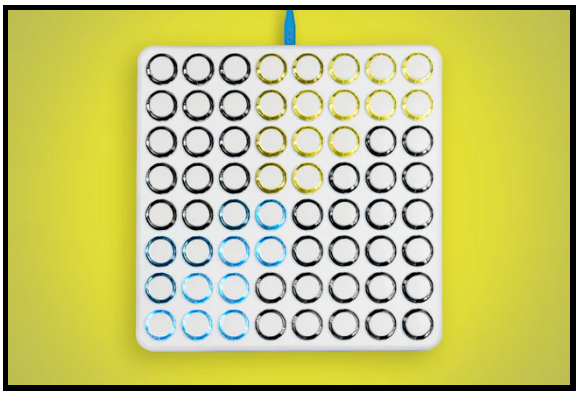
(a) (b)

(c) (d)

Figure 11: The same color palettes on the LED panels in the previous figure, presented here in a color gradient. The spread of hues is much more concentrated when the keys are mapped to the pentatonic scale. As such, it is much easier to see how the colors are differentiated between key signatures using the pentatonic mapping algorithm than it is using the chromatic mapping algorithm.

6.4 MIDI Compatibility

The system is designed primarily—if not entirely—with MIDI keyboard controllers in mind, as I believed these to be some of the most intuitive controllers for people with little familiarity with the MIDI protocol. However, MIDI controllers come in a great many forms in addition to keyboards, and while they might be more complicated to use and to program, they may also provide new avenues for music visualization. A revision of this system could be better suited for these other controllers with a series of different visualization patterns for different actuators.



(a) The Midi Fighter 64, a MIDI controller with 64 round buttons for actuators [27].



(b) The Midi Fighter 64, a MIDI controller with 16 potentiometers for actuators [28].

Figure 12: Two MIDI controllers made by the music electronics company Midi Fighter. These controllers deviate significantly from the keyboard design of other controllers, so the current version of my music visualization system may not be compatible with these controllers, However, revisions to accommodate them are certainly possible.

However, with varying controllers comes varying forms of MIDI signals. MIDI keyboards are quite simple to use because they send *one* MIDI signal each time a note is played. Other electronic instruments send multiple MIDI signals for each action, which of course would need to be processed by the system differently. This is by no means a bad thing, though; a more advanced version of this system would be fully able to accommodate these differences, appealing to a wider variety of instrumentalists.

6.5 User Feedback

Although I have reported the functionality of this system and the audiovisual experience it provides users as honestly as possible, I understand that I approach its evaluation with a certain bias as someone with a significant musical background. The most important feedback for this system's implementation would ideally come from a wide variety of other users with varying levels of musical aptitude. I originally had concepts of a plan to dedicate a portion of my design timeline to gaining feedback from other users, but later decided to spend more time on the implementation instead.

I was able to present the final prototype of my previous independent work at a public exhibit that took place in the Princeton University Makerspace in May 2024. This prototype included only the basic tactile interaction described in Chapter 4 of this report. Many participants in the exhibit seemed to enjoy using the prototype even with its limited features, and they were able to learn what it does and how to use it with only a brief introduction from me. This suggests that the system meets its goal of being intuitive to a reasonable degree.



Figure 13: A photograph of the Makerspace exhibit which displayed my independent work prototype. The prototype consisted of a MIDI keyboard controller, an Arduino Uno R3, a strip of LEDs, and my personal laptop computer to run the necessary software. As seen in the image, among the guests in attendance were students from St. Paul's School in Princeton, New Jersey, and Dr. Ruha Benjamin, the Alexander Stewart 1886 Professor of African American Studies at Princeton University.

7 Applications and Future Work

I see a number of different applications for this kind of interactive audiovisual system. The application that stood out to me the most during the design process is education. Like the examples of music visualization discussed in Chapter 2 of this report, I see potential for this technology in teaching people about musical concepts and ideas in a novel way. The use of an interactive multimodal tool can engage people to whom understanding of musical properties does not come as naturally as some others.

Beyond music education, I can also see an expanded version of this system being used to augment electronic music performance on a larger scale. For example, one could imagine a concert hall with LEDs placed throughout that show vivid animations in time with the music created by the musicians on the stage. The precedent set by artists such as deadmau5, who has used a modular cube-shaped DJ booth adorned with LEDs in his live shows since 2010, shows that this kind of multimodal performance can be widely popular [29] [30].

In a similar vein, systems like these can make music accessible to people with certain sensory impairments. Music is often portrayed as an exclusively auditory form of expression, thereby not resonating as well with those who are hard of hearing or deaf. By capturing the properties of music performance in visual, haptic, kinesthetic, or other forms of feedback, this system can democratize the medium and incentivize people who otherwise might not have originally thought it possible to participate in it.

One of the strengths of the system, in my view, is its modularity and its capacity for improvement. The MIDI protocol in the system is theoretically adaptable to many different musical scales, color palettes, and visual patterns, and it lends itself well to explorations of various musical genres and heritages.

8 Conclusion

Through this thesis, I set out to explore the expressive potential of music visualization through the lens of real-time interactivity, tactile control, and accessible design. By bridging the divide between sound and sight with a tangible, responsive interface, the system I designed can transform musical performance into a multimodal experience. The final result is a compact, modular, and intuitive platform that invites users to engage creatively with the audiovisual properties of music—regardless of their own level of musical expertise.

Although this project successfully implements a range of interactive features, it also reveals some of the constraints inherent to its hardware and software configurations. However, these limitations offer inspiration for further developments in audiovisual technology, and the existing capabilities of the system already hint at the potential of music visualization to be much more than a passive aesthetic feature.

In a broader context, this work underscores the value of combining principles of engineering, music theory, human-computer interaction, and inclusive design. It invites further research into how multimodal tools can reshape and reshape the way we learn, perform, and perceive not only music, but all other forms of human expression and creativity.

References

[1] *Geisswerks - About MilkDrop*. [Online]. Available: https://www.geisswerks.com/about_milkdrop.html (visited on 04/09/2025).

[2] *Turn on visual effects in iTunes on PC*, en. [Online]. Available: https://support.apple.com/guide/itunes/see-visual-effects-itns2947/windows (visited on 04/09/2025).

[3] L. d. studio, *Beat Saber - VR rhythm game*, en. [Online]. Available: https://beatsaber.com/ (visited on 04/09/2025).

[4] *Rhythm Heaven Groove launches on Nintendo Switch in 2026. | News & Updates | Nintendo*, en-SG. [Online]. Available: https://www.nintendo.com/sg/news/article/1diBdCj3Dabjodvp1Wz1rl (visited on 04/09/2025).

[5] *Chrome Music Lab*, en. [Online]. Available: https://musiclab.chromeexperiments.com (visited on 04/09/2025).

[6] *Project Overview ‹ Hyperscore*. [Online]. Available: https://www.media.mit.edu/projects/hyperscore/overview/ (visited on 04/09/2025).

[7] J. Rothstein, *MIDI: A Comprehensive Introduction*, en. A-R Editions, Inc., Jan. 1995.

[8] F. R. Moore, "The Dysfunctions of MIDI," *Computer Music Journal*, vol. 12, no. 1, pp. 19–28, 1988, Publisher: The MIT Press. DOI: 10.2307/3679834. [Online]. Available: https://www.jstor.org/stable/3679834 (visited on 04/10/2025).

[9] S. Smith and G. Williams, "A visualization of music," in *Proceedings. Visualization '97 (Cat. No. 97CB36155)*, Oct. 1997, pp. 499–503. DOI: 10.1109/VISUAL.1997.663931. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/663931 (visited on 04/10/2025).

[10] C. Schierle, "Visual MIDI Data Comparison," 2020. [Online]. Available: https://www.semanticscholar.org/paper/Visual-MIDI-Data-Comparison-Schierle/413c997ca6821d7ab4c2d19a2c5f160bc7f6ece7 (visited on 04/10/2025).

[11] I. B. Milošević, Ž. M. Bogavac, D. Regodić, and B. Milošević, "Visualization of Music - MIDI Content," in *2022 57th International Scientific Conference on Information, Communication and Energy Systems and Technologies (ICEST)*, Jun. 2022, pp. 1–4. DOI: 10.1109/ICEST55168.2022.9828738. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/9828738 (visited on 04/10/2025).

[12] E. Edmonds, G. Turner, and L. Candy, "Approaches to interactive art systems," in *Proceedings of the 2nd international conference on Computer graphics and interactive techniques in Australasia and South East Asia*, ser. GRAPHITE '04, New York, NY, USA: Association for Computing Machinery, Jun. 2004, pp. 113–117. DOI: 10.1145/988834.988854. [Online]. Available: https://dl.acm.org/doi/10.1145/988834.988854 (visited on 04/10/2025).

[13] R. Kluszczynski, "Strategies of interactive art," *Journal of Aesthetics & Culture*, vol. 2, no. 1, p. 5525, Jan. 2010, Publisher: Routledge _eprint: https://doi.org/10.3402/jac.v2i0.5525. DOI: 10.3402/jac.v2i0.5525. [Online]. Available: https://doi.org/10.3402/jac.v2i0.5525 (visited on 04/10/2025).

[14] *Rain Room | LACMA*, en, Jan. 2022. [Online]. Available: https://www.lacma.org/art/exhibition/rain-room (visited on 04/10/2025).

[15] *Six-Forty by Four-Eighty*, en-US. [Online]. Available: http://jamiezigelbaum.com/six-forty-by-four-eighty (visited on 04/10/2025).

[16] T. Blaine and T. Perkis, "The Jam-O-Drum interactive music system: A study in interaction design," in *Proceedings of the 3rd conference on Designing interactive systems: processes, practices, methods, and techniques*, ser. DIS '00, New York, NY, USA: Association for Computing Machinery, Aug. 2000, pp. 165–173. DOI: 10.1145/347642.347705.

[Online]. Available: https://dl.acm.org/doi/10.1145/347642.347705 (visited on 04/10/2025).

[17] *Arturia - KeyStep 37*, fr-FR. [Online]. Available: https://www.arturia.com/fr/products/hybrid-synths/keystep-37/overview (visited on 04/09/2025).

[18] *Arturia - BeatStep*, fr-FR. [Online]. Available: https://www.arturia.com/fr/products/hybrid-synths/beatstep/overview (visited on 04/09/2025).

[19] *Harmonizing Sound and Light: MIDI-Enabled LED Visualization System, Kalu James Christopher Obasi, UG '25 (704C3684)*, en. [Online]. Available: https://mediacentral.princeton.edu/media/Harmonizing+Sound+and+Light%3A+MIDI-Enabled+LED+Visualization+System%2C+Kalu+James+Christopher+Obasi%2C+UG+25+%28704C3684%29/1_r0l1tu80 (visited on 04/10/2025).

[20] *UNO R3 | Arduino Documentation*. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3/ (visited on 04/09/2025).

[21] *Mega 2560 Rev3 | Arduino Documentation*. [Online]. Available: https://docs.arduino.cc/hardware/mega-2560/ (visited on 04/09/2025).

[22] *Due | Arduino Documentation*. [Online]. Available: https://docs.arduino.cc/hardware/due/ (visited on 04/09/2025).

[23] *UNO R4 Minima | Arduino Documentation*. [Online]. Available: https://docs.arduino.cc/hardware/uno-r4-minima/ (visited on 04/09/2025).

[24] *UNO R4 WiFi | Arduino Documentation*. [Online]. Available: https://docs.arduino.cc/hardware/uno-r4-wifi/ (visited on 04/09/2025).

[25] *The Hairless MIDI<->Serial Bridge*. [Online]. Available: https://projectgus.github.io/hairless-midiserial/ (visited on 04/09/2025).

[26] *loopMIDI | Tobias Erichsen*, en-US. [Online]. Available: https://www.tobias-erichsen.de/software/loopmidi.html (visited on 04/09/2025).

[27] *Midi Fighter 64*. [Online]. Available: https://www.midifighter.com/shop/midi-fighter-64 (visited on 04/10/2025).

[28] *Midi Fighter Twister*. [Online]. Available: https://www.midifighter.com/shop/midi-fighter-twister (visited on 04/10/2025).

[29] *Deadmau5 To Take His Brand-New "Cube V3" Show On Tour This Fall | GRAMMY.com*. [Online]. Available: https://grammy.com/news/deadmau5-take-his-brand-new-cube-v3-show-tour-fall (visited on 04/10/2025).

[30] *Cubev3.com*, en. [Online]. Available: https://cubev3.com/ (visited on 04/11/2025).

A Appendices

A.1 Code

The code used on the Arduino has been made available in a GitHub repository, in addition to a "readme" document with instructions for setup and usage. Here is the link to the repository: `https://github.com/Kalu-JC-Obasi/PlayThePlatform`

A.2 Photos and Videos

I took many photos and videos of the system during its development to demonstrate its functionality. Many of the system's features are much more easily *shown* than *told*, so these photos and videos have all been made available in a Dropbox repository, in addition to a "readme" document with more context on each video. Here is the link to the repository: `https://tinyurl.com/3t4pz9st`

B Honor Code

This paper represents my own work in accordance with University regulations. /s/ Kalu Obasi